Para o Wilson: a app nas lojas e o backend das compras

Olá Wilson,

Estamos a transformar o site numa **app para o Google Play e a App Store**. A parte do frontend e a do backend já estão programadas e testadas, mas **ainda não estão em produção nem commitadas**. Preciso da tua ajuda no backend para avançarmos. Em baixo está o que fizemos e, na secção 3, os passos que são teus.

— Lukeny, 2026-10-30

---

1. O que estamos a fazer

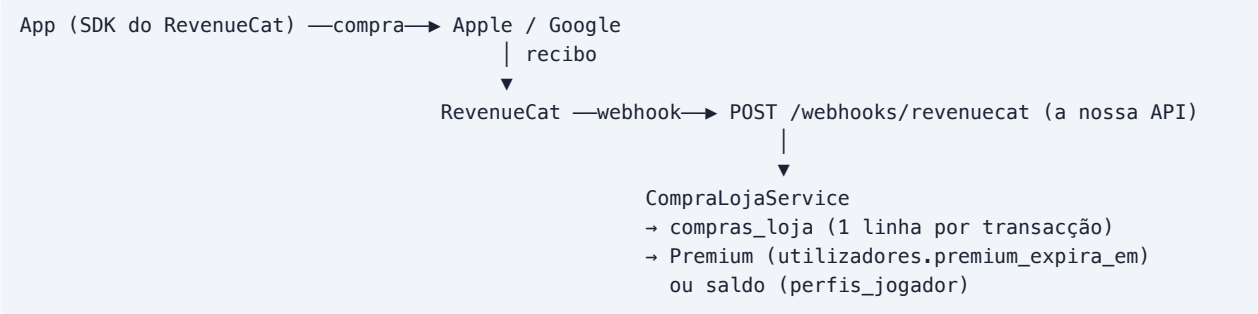
A app é o próprio site dentro de uma "casca" nativa (Capacitor). A app abre `https://www.janelasparaalma.com`; não leva uma cópia do build lá dentro. Assim:

- o `/api/*` continua na mesma origem, e o cookie `httpOnly` de sessão funciona como no site;
- cada deploy do Vercel chega logo à app, sem publicar versão nova nas lojas.

Os projectos nativos estão em `frontend/android` e `frontend/ios`, e a configuração em `frontend/capacitor.config.ts`.

O problema dos pagamentos. A Apple e a Google não aceitam que, dentro de uma app, se vendam produtos digitais (o Premium, os diamantes e as moedas do jogo) por transferência + comprovativo. Decidi que **dentro da app esses produtos se compram pela App Store / Google Play**. No site, nada muda: continua a transferência e o comprovativo.

Como funciona:



- **Quem dá o Premium ou o saldo é só a API**, quando o RevenueCat chama o webhook. O frontend nunca credita nada: depois de comprar, fica a ler a API até a compra aparecer.
- A app identifica-se no RevenueCat com o `id` da conta (`app_user_id` = `utilizadores.id`). É assim que o webhook sabe a quem pertence a compra.

2. O que mudou no backend (para rever)

> Toca o **esquema da base de dados** e o **paywall**, por isso precisa da tua revisão antes do merge (CLAUDE.md §9, ponto 4).

Ficheiro	O quê
`api/alembic/versions/d4b8f2a6c1e3_compras_loja.py`	Tabela nova `compras_loja`. Índice único `(loja, transacao_id)`; `utilizador_id` com `ON DELETE SET NULL` (é um registo financeiro, sobrevive à conta)
`api/app/repositories/orm_models.py`	Modelo `CompraLoja`

`api/app/repositories/compras_loja_repository.py`	Grava a compra e aplica o efeito **na mesma transacção** (`INSERT ... ON CONFLICT DO NOTHING`); faz o reembolso
`api/app/services/compra_loja_service.py`	As regras (abaixo) e o catálogo de produtos
`api/app/routers/webhooks.py` + `api/app/schemas/webhooks.py`	`POST /webhooks/revenuecat`
`api/app/core/dependencies.py`	`obter_webhook_revenuecat_valido`: compara o `Authorization` com `REVENUECAT_WEBHOOK_SEGREDO` usando `hmac.compare_digest`. Sem segredo configurado, recusa sempre (a mesma regra de `obter_cron_valido`)
`api/app/core/config.py`	`revenuecat_webhook_segredo` (vazio por omissão) e `revenuecat_aceitar_sandbox` (`true` por omissão)
`api/app/main.py`	Regista o router
`infra/gcloud/04-deploy.sh`	Se o segredo `jpa-revenuecat-webhook` existir, liga-o como `REVENUECAT_WEBHOOK_SEGREDO`
`api/.env.example`	As duas variáveis novas
Testes	`tests/services/test_compra_loja_service.py`, `tests/routers/test_webhooks_router.py`, `tests/integracao/test_compras_loja_pg.py`

Regras do `CompraLojaService` (vale a pena confirmar se concordas):

- Eventos que dão alguma coisa: `INITIAL_PURCHASE`, `RENEWAL` e `NON_RENEWING_PURCHASE`.
- **Premium:** soma o período pago (`expiration_at_ms − purchased_at_ms`) à validade actual, com **tecto de 35 dias por transacção**. Sem datas no evento, dá 30 dias. Não mexe nas 5 leituras de validade que já existem: é o mesmo modelo do comprovativo aprovado (+30 dias somados).
- **Diamantes e moedas:** a quantidade vem de `PACOTES_DIAMANTES` / `PACOTES_MOEDAS`, nunca do evento.
- **Reembolso** (`CANCELLATION` com `cancel_reason = CUSTOMER_SUPPORT`): tira só o que essa transacção deu. No Premium, se não sobrar validade, desliga-o; no saldo do jogo, usa `GREATEST(saldo − q, 0)`, para nunca ficar negativo.
- **Ignorados, sempre com 200** (senão o RevenueCat repete): eventos sem interesse (`EXPIRATION`, `TEST`, `TRANSFER`...), produto desconhecido (fica no log como **erro**), conta inexistente ou anonimizada, loja que não seja a Apple nem a Google.
- **Repetido:** a mesma transacção devolve `"repetida"` e não aplica nada.
- **Sandbox conta**, porque o revisor da Apple compra em sandbox na app de produção. `REVENUECAT_ACEITAR_SANDBOX=false` desliga isto.

Ids dos produtos (têm de ser iguais nas lojas, no RevenueCat e no código): `jpa_premium_mensal` (no Google Play chega como `jpa_premium_mensal:mensal`), `jpa_diamantes_pequeno|medio|grande` e `jpa_moedas_pilha|saco|bau`.

Estado dos testes: a API tem 1136 testes a passar e o `ruff check app tests` está limpo. Os 4 testes novos de integração passaram contra um Postgres real. O SQL da migração nova foi validado em Postgres, mas **a cadeia completa do `alembic upgrade head` não correu localmente** (o Postgres que usei não tinha `pgcrypto`). O CI faz essa verificação.

3. Os teus passos

Passo 1 — Rever e juntar o código

1. O código está na branch `app/lojas-compras-nativas`, feita a partir do `main` (só a preparação da app, sem o trabalho da página inicial nem do Kamba Mensal):

   ```
   git fetch origin && git switch app/lojas-compras-nativas
   ```

   Abre o PR para o `main` a partir dela (ou diz-me, que eu abro).

2. **Ordem das migrações:** `d4b8f2a6c1e3` (compras) vem logo a seguir à head actual do `main`, `b4e7d2a9c6f1`. A migração do Kamba Mensal (`c7a3e1f9d2b4`, ainda não commitada) vai entrar **depois** desta, já ajustada para isso na minha cópia local. Confirma com `python -m alembic heads`: tem de haver **uma só** head.
3. Rever as regras da secção 2. Se discordares de alguma, diz-me antes do merge.
4. Confirmar que o CI passa: `ruff`, `pytest` e `alembic upgrade head` contra um Postgres real.

Passo 2 — Testar o webhook localmente

1. Acrescenta ao serviço `api` do `docker-compose.yml`, só no teu ambiente local:

   ```
   REVENUECAT_WEBHOOK_SEGREDO: segredo-local
   ```

2. Sobe a API, migra e cria a conta de teste:

   ```
   docker compose up -d
   ```

   ```
   docker compose exec api python -m alembic upgrade head
   ```

   ```
   docker compose exec api python -m scripts.dados_teste_locais
   ```

3. Tira o `id` da conta de teste:

   ```
   docker compose exec db psql -U jpa -d jpa -c "select id, email from utilizadores;"
   ```

4. Simula uma compra do Premium (troca `<ID>` pelo id da conta):

   ```
   curl -s -X POST http://localhost:8000/webhooks/revenuecat -H "Authorization: segredo-local" -H "C
   ontent-Type: application/json" -d '{"event":{"type":"INITIAL_PURCHASE","app_user_id":"<ID>","stor
   e":"APP_STORE","environment":"SANDBOX","product_id":"jpa_premium_mensal","transaction_id":"teste-
   1","purchased_at_ms":1793361600000,"expiration_at_ms":1795953600000}}'
   ```

   Esperado: `{"resultado":"aplicada"}`. Repete o mesmo pedido e tem de dar `"repetida"`. Confirma em `utilizadores.premium_expira_em`.

5. Simula o reembolso, com o mesmo `transaction_id`: `"type":"CANCELLATION","cancel_reason":"CUSTOMER_SUPPORT"`. Esperado: `"reembolsada"`, e o Premium volta ao que era.
6. Sem o cabeçalho, ou com um valor errado, tem de dar **403**.

Passo 3 — Produção (depois do merge)

1. Gerar um segredo longo e aleatório e guardá-lo no Secret Manager. **Não corras o `03-secrets.sh` inteiro** (CLAUDE.md §4.8: pode gerar um `JWT_SECRET_KEY` novo e desligar toda a gente):

   ```
   python3 -c 'import secrets; print(secrets.token_urlsafe(48))'
   ```

```
printf '%s' 'O-SEGREDO-GERADO' | gcloud secrets create jpa-revenuecat-webhook --data-file=-
```

2. Dar acesso ao segredo à conta de serviço do Cloud Run, como o `03-secrets.sh` faz para os outros segredos:

```
PROJECT_NUMBER="$(gcloud projects describe "$PROJECT_ID" --format='value(projectNumber)')"
gcloud secrets add-iam-policy-binding jpa-revenuecat-webhook --member="serviceAccount:${PROJECT_N
UMBER}-compute@developer.gserviceaccount.com" --role="roles/secretmanager.secretAccessor"
```

3. Fazer deploy da API (automático ao fazer merge, ou `04-deploy.sh`). No log do deploy **não** pode aparecer "Sem jpa-revenuecat-webhook".
4. **Passa-me o segredo por um canal privado** (nunca no Git, no chat do grupo nem num PR). Sou eu que o ponho no RevenueCat, como valor do *Authorization header* do webhook.
5. Confirmar que o webhook chega à API pelo rewrite do Vercel:

```
curl -s -o /dev/null -w '%{http_code}\n' -X POST https://www.janelasparaalma.com/api/webhooks/rev
enuecat
```

   Esperado: **403** (sem segredo). Um 404 quer dizer que o rewrite ou o router não estão bem.

6. Quando eu ligar o webhook no RevenueCat, carrego em "Send test event". No log do Cloud Run tem de aparecer o pedido com **200** e `"resultado":"ignorada"` (o evento `TEST` não dá nada).

Passo 4 — Depois de as compras arrancarem

- Vigiar no log do Cloud Run: **"compra na loja com produto desconhecido"**. Quer dizer que alguém pagou um produto que não está no catálogo e não recebeu nada. Corrige-se acrescentando o produto ao `compra_loja_service.py`.
- Vigiar também **"conta inexistente ou anonimizada"**.
- Um pacote novo na loja do jogo precisa de entrar **também** no catálogo das lojas (Apple, Google e RevenueCat) com o id `jpa_diamantes_<id>` / `jpa_moedas_<id>`.

4. O que fica do meu lado (só para saberes)

- Contas de programador na Apple (99 USD/ano) e na Google (25 USD), D-U-N-S e contratos de pagamento das lojas. **Primeiro confirmo que as lojas pagam a uma conta em Angola**: se não pagarem, as compras na app ficam suspensas.
- Preços dos produtos (definem-se nas consolas das lojas, não no código).
- Criar os produtos, a conta no RevenueCat e as chaves públicas no Vercel (`VITE_REVENUECAT_CHAVE_IOS` / `_ANDROID`).
- Compilar e submeter as apps (Android Studio e Xcode).

Guia completo, com custos e privacidade: `docs/APP_LOJAS.md`. Visão geral da arquitectura: `CLAUDE.md` §2 ("App nativa").

Qualquer dúvida, fala comigo. Obrigado!